



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

POROČILO PRI PREDMETU RAČUNALNIŠKA GRAFIKA

Zadnja različica aplikacije za 3D interaktivno vizualizacijo

Avtorji:

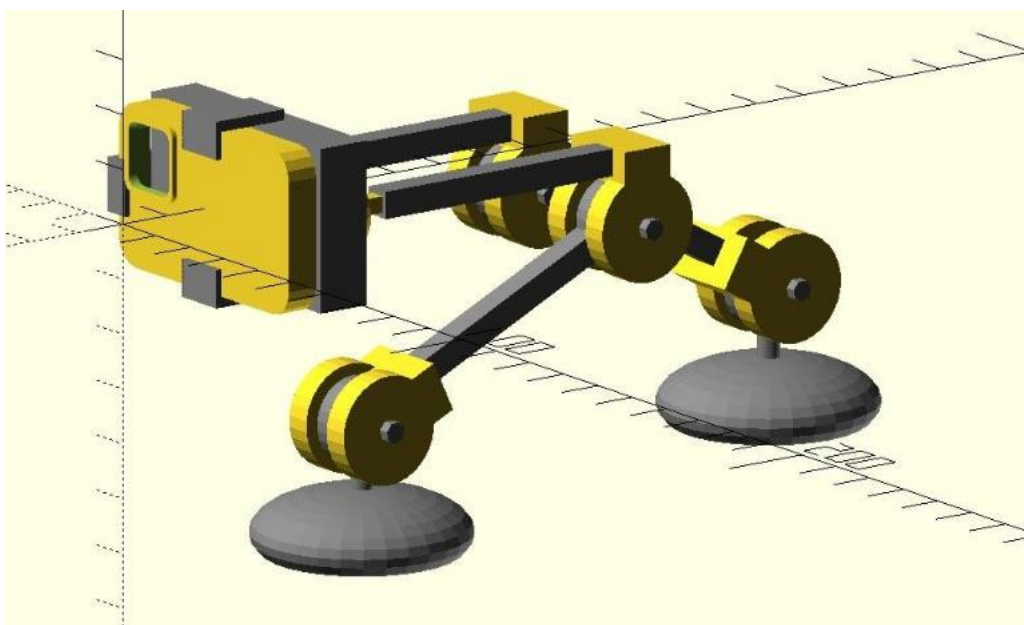
Kjara Haložan,

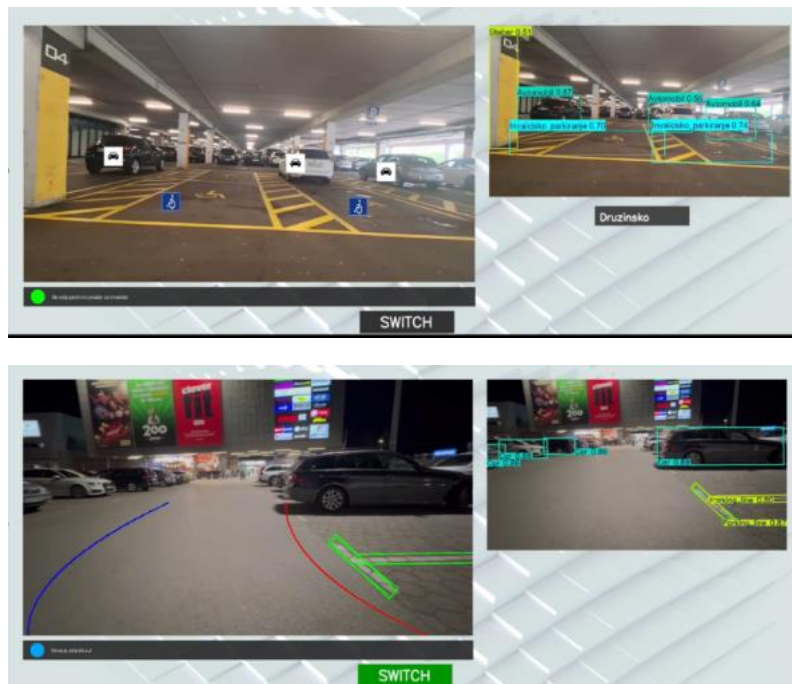
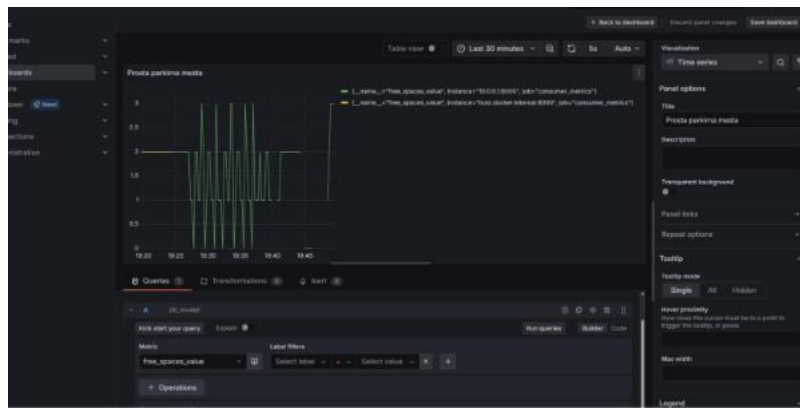
Nejla Perenda,

1. FUNKCIONALNOSTI

Naša aplikacija do sedaj omogoča zaznavo različnih parkirnih mest . Ob tem sistem samodejno zaznava in določi za kateri tip parkirne površine gre, torej ali je parkirišče standardno, za invalide, za družine, električne avtomobile.. Ob tem uporabniku sporoča, ali je parkirno mesto prosto in ali ustreza vozilu (glede na tip)

Prav tako omogoča zaznavo parkirnih črt in ovir na parkirišču, s tem zazna ali se lahko avtomobil parkira. Pove tudi kako daleč od avtomobila je ovira. Zadnja funkcionalnost so tudi Bezierjeve krivulje, ki služijo kot pomoč, pri zavojih v parkirišče.





Izdelki pri posameznih predmetih:

Pri predmetu **Umetna inteligenca** smo naučili 3 modele, ki sodelujejo med sabo:

- Model za zaznavo črt
- Model za krivulje
- Model za parkirna mesta

Pri predmetu **Signali in slike** smo izvedli augmetnacio in pripravili učni material za učenje ai.

Pri predmetu **Uvod v računalniško geometrijo** smo izdelali 3D model držala za telefon/kamero.

Pri predmetu **Sistemska programska oprema** smo izdelali TCPpovezavo, grafano, VPN,...

Katere podatke prejemamo in obdelujemo

Prejeti podatki so videoposnetki, kateri zaznavajo katerega tipa parkirišča so naša mesta in ali so le ta prosta/zasedena. Torej podatki o samem parkirišču. Prav tako pa prejemamo pozicije črt na parkirišču in oddaljenosti od ovir.

Prikaz v 3D okolju

Imeli bomo model, ki se bo na podlagi senzorja oddaljenosti in kamere samostojno prakiral v parkirno mesto.

Iz senzorjev bomo razbrali oddaljenost, sliko kamere, ...

Uporabnik bo lahko na 3d projekciji videl okolico okoli sebe in nato izbral parkirno mesto v katerega se avto želi parkirati.

Uporabniki bi na projekciji vidil enostavno okolico, kjer bi se prikazali le objekti kot so:

- Človek
- Avtomobili
- Prosta parkirna mesta
- Invalidski parking
-



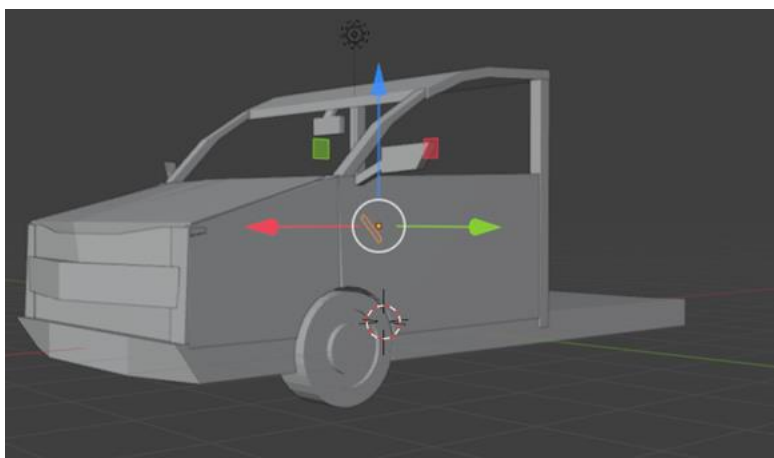
2. 3D MODEL– izdelava osnovne oblike

Nejla

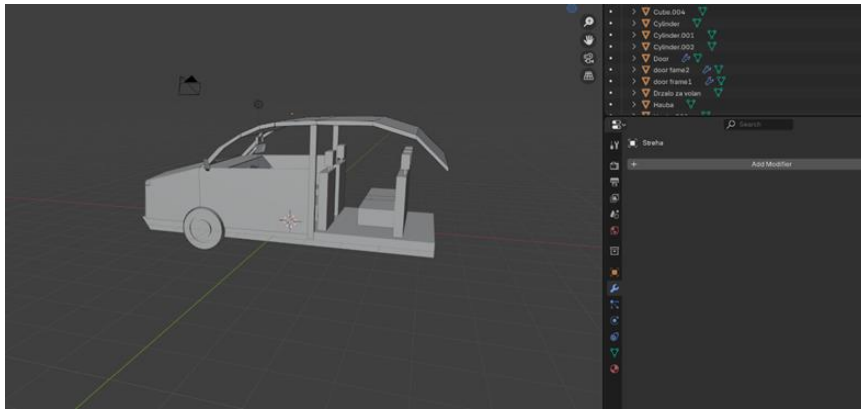
Moja naloga je bila priprava armaturne plošče, oziroma notranji prostor avtomobila, s katerega bomo kasneje tudi simulirali delovanje aplikacije. Delo sem si malo utežila, saj nisem vedela kako naj se lotim modela, zato sem naredila celoten model avtomobila za nadaljne simulacije, s POUDARKOM na armaturni plošči in notranjosti.

Prvo sem uporabila cube, za dno avtomobila, haubo in luči. S pomočjo edit mode, sem izbrala stranice, in jih zaoblila, da dobi model bolj realističen izgled. Za pokrov motorja sem uporabila cube, katerega sem iustrezno skalirala in nastavila na desno stran, zarotirala in nato s pomočjo funkcije 'mirror', zrcalila glede na dno avtomobila, tako da izgleda lepše in realistično.

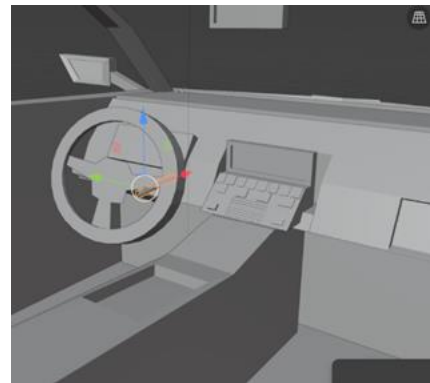
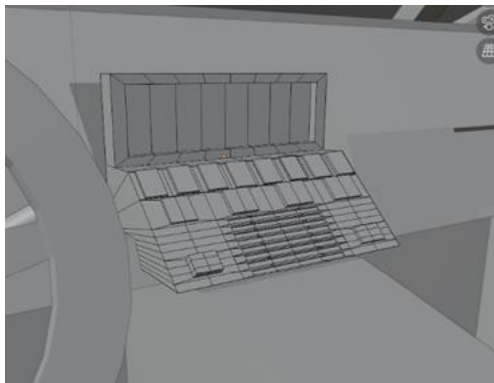
Nato sem s cube naredila še vrata in ogledala, katere sem prav tako zrcalila glede na dno avtomobila. V Zrcalih sem v edit mode, ponovno raztegnila stranice, nato pa izbrala celotno ploskev in s pomočjo 'ctrl + I' in 'ctrl + E' naredila vdolbine za steklo. Prav tako sem na podoben način naredila vzvratno ogledalo. Nato sem s cube naredila stebričke in streho, s tem da sem streho 'rezla' da sem jo lahko bolj zaobljeno položila. Prav tako sem s pomočjo cilindra ustvarila gume, naredila sem bolj zivahno obliko s pomočjo 'ctrl + I' in 'ctrl + E', da da pravi efekt gume, nato pa jo prezrcalila



Na tej točki, bi se lahko ustavila in naredila armaturo, a sem se odločila nadaljevati, zato sem nadaljevala s streho in dodala sedeže, kateri so narejeni iz cube. Sprednji sedeži so prezrcaljeni, zadnji pa so duplicirani in postavljeni skupaj.



Nato sem se odločila nadaljevati na armaturni plošči, katera je iz cube, ploskev je razrezana in razpotegnjena, tako da deluje bolj zaobljeno. Nato sem vzela cube, in s pomočjo le tega naredila 'luknjo'/'votlino' s pomočjo Booleanove operacije za razliko. Ekranček je mali cube z ureznino, tipke in klima pa drugi, bolj razpotegnjen cube, tekstura/oblika pa je ponovno narejena iz ctrl + I in 'ctrl + E'.

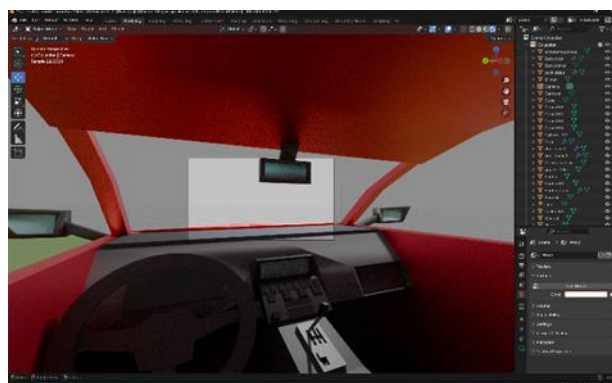
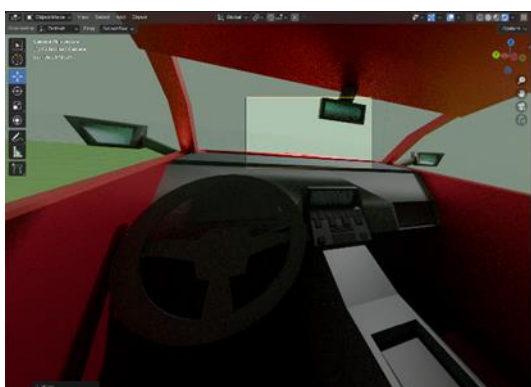


Nato sem naredila še volan s pomočjo cilindra ki se ga skalirala in rotirala, nato sem pa luknje naredila s pomočjo funkcije 'cut'. Prav tako sem dodala paličice za brisalce in svetlobna opozorila. Tudi ti so narejeni iz cilindrov. Naredila sem še konzolo iz cube. Za volanom je še dejanska 'armaturna plošča', ki prikazuje hitrost in porabo, prav tako narejena iz cube.

Na koncu sem naredila še zadnja vrata, kljuke in prtljažnik, vse iz kocke, vse to je prezrcaljeno. Prav tako sem duplicirala gume in jih potegnili nazaj. S pomočjo večjega cilindra sem izrezala obliko ven iz trupa in vrat avtomobila.



Na koncu sem dodala še barve, uporabila sem rdečo(zunanost), sivo(notranjost), oranžno(luči), in modro(ogledala). Kamero sem postavila na sedež, da lahko dejansko simuliramo vožnjo. Prav tako se mi je zdelo, kot da nekaj manjka, zato sem se lotila še menjalnika.



Menjalnik sem naredila iz cube, zareze sem naredila s pomočjo funkcije 'cut', prav tako sem uporabila cylinder za dejanski menjalnik, in razrez za gumb za ročno zavoro. S tem je pa moje delo končano. Končni rezultat izgleda tako.



Na koncu sem pa še dodala teksturo na sedeže:

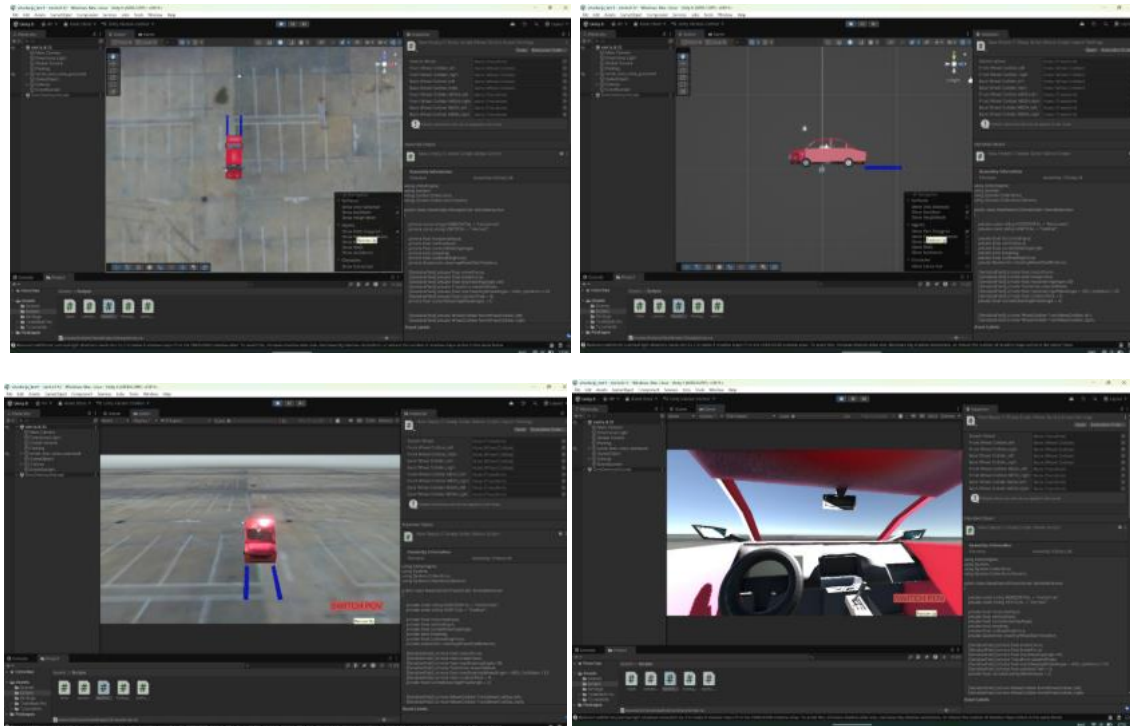


3. PRVA RAZLIČICA APLIKACIJE

Nejla: Priprava simulacije vzratnega parkiranja

Pripravila sem temelje za simulacijo vzratnega parkiranja, na kateri bo prikazano premikanje avtomobila, pod kontrolo uporabnika.

Uporabila sem moj bledner model avtomobila in ga postavila v Unity, prav tako pa naredila tla in jim dala teksturo oziroma material parkirišča.. Izbrala sem objekte koles in dodala 'colliderje', prav tako pa postavila avto v box. Ko sem vse uredila, sem se lotila pisanja kod in funkcionalnosti. Imam 4 uporabljene programe, en je za kontroliranje avtomobila in pravilno obnašanje le tega, v njej sem omogočila tudi animacijo volana, ki nakazuje realne premike za zavoje. Drugi program je za zamenjavo pogleda iz ptičje perspektive v perspektivo voznika, za lažjo preglednost. Tretji program omogoča sledenje kameri iz ptičje perspektive, da nam avto ne pobegne izpred oči, ter četrti, ki omogoča izris krivulj, ki nakazujejo zavoj avtomobila.



Lovro Obreza: Priprava simulacije okolja

Ustvaril sem projektno strukturo (frontend + backend) in dodal datoteke index.html, index.js, controls.js in strežnik server.py + main.py.

Na frontendu sem uporabil Three.js, OBJLoader in MTLLoader, na backendu pa FastAPI, WebSocket, YOLO in OpenCV.

Implementiral sem renderer, sceno, kamero, osvetlitev in tla. Kamera je postavljena tako, da gleda na center scene (avto). Kamera uporablja spherical koordinate in vedno gleda v target. Dodal sem modele: avto, parkirno mesto, steber, človek in implementiral pomožne funkcije preloadModel() in cloneObject() za pravilno in hitro nalaganje in kloniranje modelov.

Pridobivam YOLO detekcije in iz njih generiram 3D objekte. Ob vsakem sporočilu pobrišem prejšnje objekte in narišem nove. Pozicije modelov interpoliram glede na podatke detekcije.

Implementiral sem lastno kontrolo kamere (controls.js):

- levi klik: rotacija pogleda,
- sredinski klik: horizontalno premikanje kamere,
- scroll: zoom.

Implementiral sem WebSocket povezavo na ws://localhost:8000/ws. Backend pošilja YOLO detekcije v realnem času. Aplikacija prikazuje rezultate detekcij kot 3D objekte. Objekti se

premikajo po Z-osi, kar simulira premikanje naprej – tukaj manjkajo kontrole avtomobilov. Backend lahko deluje v realnem času ali prebira iz yolo_data.txt.

Kjara Haložan: Priprava komunikacije

S pomočjo MQTT Mosquitto sem ustvarila broker, ki posluša na naslovu 192.168.89.218:1883. Program »publisher« vzpostavi povezavo z njim in vsake 0,1 sekunde pošilja podatke o rotaciji (sensor/rotation) in vertikalnem gibanju (sensor/vertical). Vertikalno gibanje se nadzoruje s tipkama w in s, medtem ko se kot rotacije naključno spreminja do ± 10 % znotraj intervala od -1 do 1. Za lažje testiranje »publisherja« v terminalu lahko zaženemo ukaz `docker exec -it mosquitto mosquitto_sub -t sensor/rotation`, ki prikazuje poslana sporočila na temo "sensor/rotation".

Program »subscriber« se poveže na isti broker in se naroči na teme sensor/rotation in sensor/vertical. Ob prejemu sporočila pretvori vsebino v številko in jo izpiše v terminal skupaj z imenom teme, ob neveljavnih podatkih pa izpiše opozorilo.

Ker smo naleteli na precej težav z nameščanjem in uporabo MQTT knjižnice v okolju Visual Studio in Unity, smo se zaenkrat odločili, da bomo podatke pošiljali preko TCP protokola. Za to sem izdelala posebno skripto, ki po TCP protokolu na portu 5005 pošilja podatke vsakih 0,1 sekunde, pri čemer se prenašajo horizontalni in vertikalni podatki.

V kodi za simulacijo vzratnega parkiranja sem dodala tudi funkcionalnost, ki preko TCP povezave prejema podatke o premikanju vozila naprej in nazaj (vertikalno) ter o premikanju parkirnih črt oziroma pnevmatik levo in desno (horizontalno).

4. DRUGA RZLIČICA APLIKACIJE

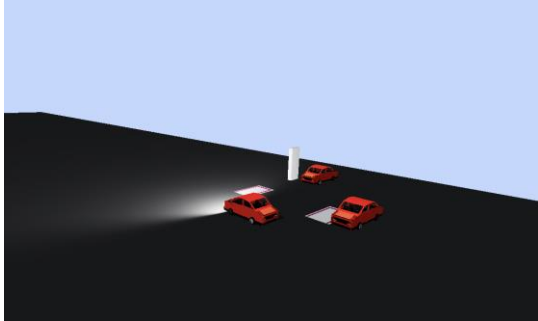
Lovro:

Light mode (dnevni način)

V svetlem načinu je scena nastavljena za simulacijo dnevnih svetlobnih pogojev:

- Uporabljen je DirectionalLight z visoko intenziteto, ki simulira sonce. Svetloba oddaja paralelne žarke in omogoča realistične, ostre sence.
- Shadow mapping je omogočen (PCF shadow map) z višjo resolucijo shadow map, kar izboljša kakovost senc na tleh in objektih.
- Dodan je AmbientLight z nizko intenziteto, ki prepreči popolnoma črne sence.
- HemisphereLight zagotavlja barvni kontrast med nebom in tlemi ter mehkejši "bounce light".
- Scena ima svetlo ozadje (sky color), megla je onemogočena, kar zagotavlja jasno vidljivost na daljše razdalje.

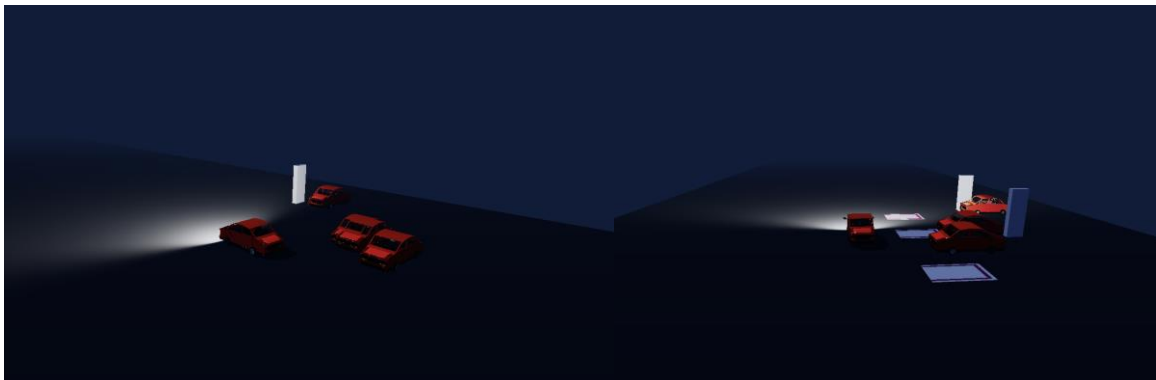
- Aktiviran je environment mapping s pomočjo CubeCamera, kar omogoča osnovne odboje svetlobe na materialih.
- Tone mapping (ACES Filmic) ima višji exposure, prilagojen dneveni svetlobi.



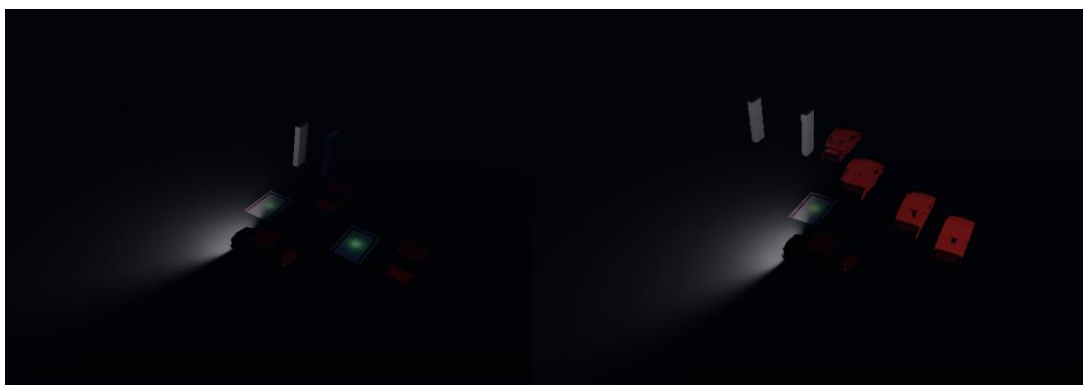
Dark mode (nočni način)

Temni način je zasnovan za simulacijo nočne vožnje in parkiranja:

- Ozadje scene je nastavljeno na zelo temno barvo, ki simulira nočno nebo.
- Globalna osvetlitev je močno zmanjšana
- DirectionalLight deluje kot šibka lunina svetloba z nizko intenziteto.
- AmbientLight je minimalen, dodan zgolj zato, da materiali ne izgubijo osnovne vidnosti (PBR stabilnost).
- HemisphereLight ima zelo nizko intenziteto in hladen barvni ton.
- Dodana je FogExp2, ki simulira nočno meglo in zmanjšuje vidljivost oddaljenih objektov, hkrati pa poveča občutek globine scene.
- Environment map je temnejši, kar zmanjša svetle odboje in ohranja nočni kontrast.
- Tone mapping ima nižji exposure, kar prepreči "pregorevanje" svetlih elementov v temi.



Še močnejša tema (manj uporabno za samo aplikacijo)



Parking lights (osvetlitev parkirnih mest)

Za boljšo orientacijo v temnem načinu so bila parkirna mesta dodatno poudarjena:

- Vsako parkirno mesto ima dodan PointLight, postavljen tik nad površino parkirišča.
- Svetloba ima omejen doseg in višjo intenziteto, kar jasno označi prosta parkirna mesta.
- Glow efekt ostane viden tudi ob zelo nizki globalni osvetlitvi, kar izboljša uporabnost nočnega načina.

Osvetlitev zaznanih objektov (vozila, ovire)

- Zaznani avtomobili imajo v dark mode dodane svetlobne poudarke (PointLight + emissive), da so jasno ločeni od okolja.
- To prepreči, da bi se objekti "izgubili" v temi, in izboljša zaznavanje ovir.
- Svetlobni poudarki so lokalni in ne vplivajo na celotno sceno, kar ohranja realističen kontrast.

Luči vozila

Avtomobil ima implementirane žaromete (SpotLight):

- Širok svetlobni stožec simulira realne avtomobilske luči.
- Luči so usmerjene preko target objektov, vezanih na avtomobil.
- Žarometi osvetlujejo tla in objekte pred vozilom ter dodatno izboljšajo orientacijo v dark mode.

Preklapljanje med načini

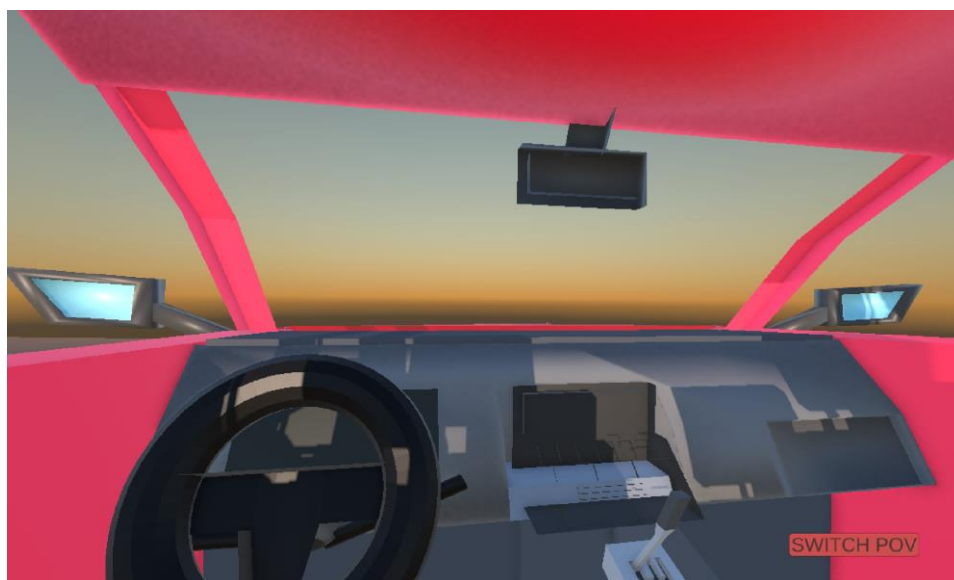
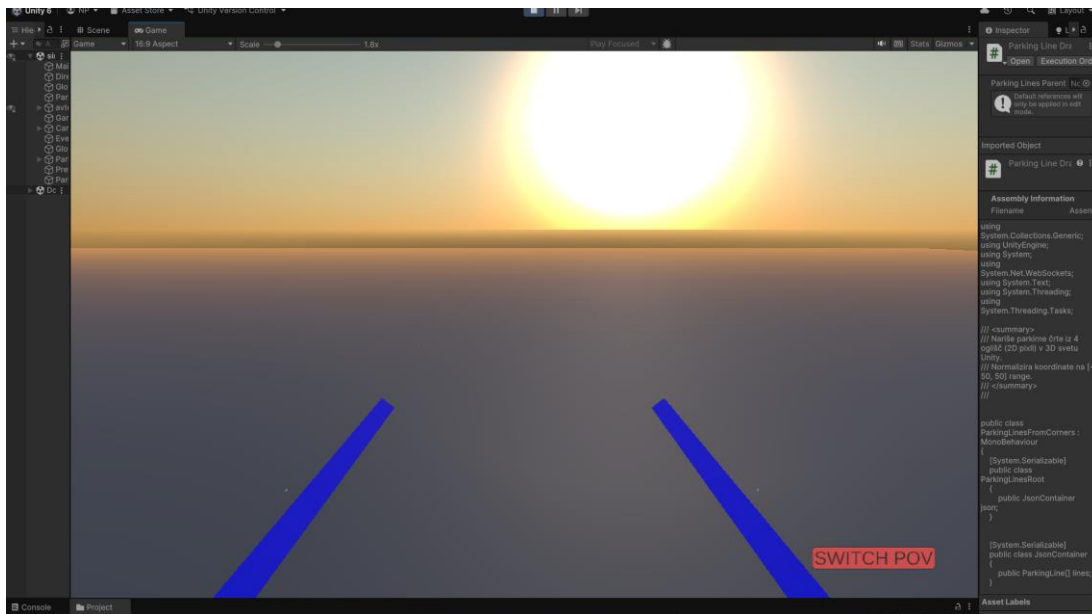
- Ob preklopu se vse obstoječe luči in svetlobni objekti odstranijo.
- Ponovno se inicializirajo luči, megla, environment map in tone mapping glede na izbran način.

- Environment kamera se posodobi, da so odboji skladni z novo osvetlitvijo.
- Preklop ne zahteva ponovnega nalaganja scene ali modelov.

Nejla:

Dodan efekt sočnih žarkov:

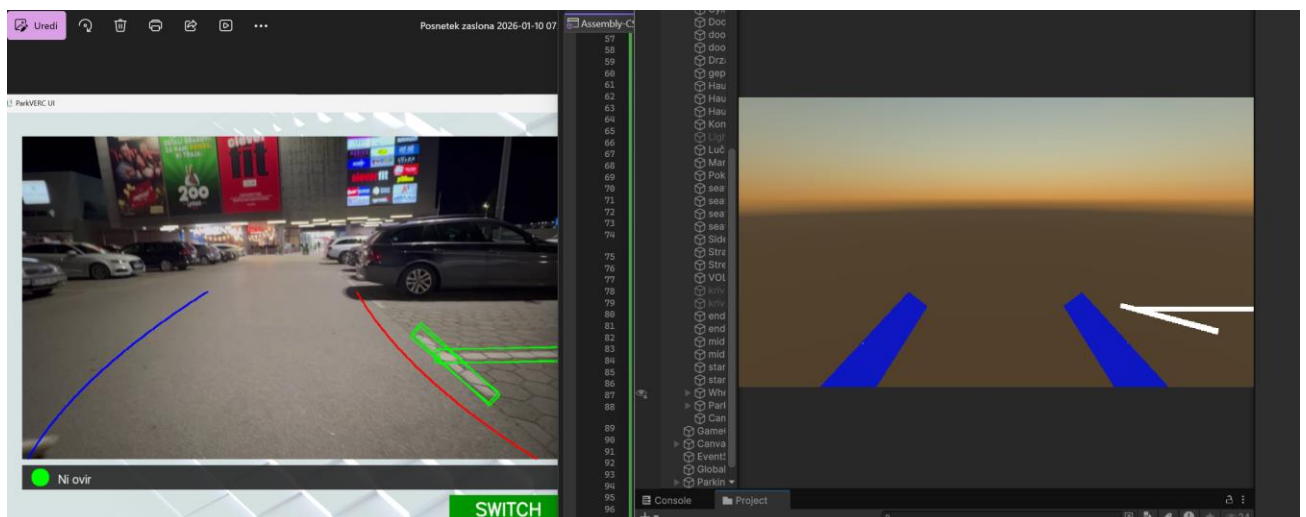
V unity sem s pomočjo Directional lighting dodala in posodobila zunanjo luč, torej sonce, kot dodaten efekt k simulaciji. Directional Lighting sem dodala močno intenziteto, spremenila barve in atmosfero, prav tako pa premaknila luč (sonce) bolj nizko, da ga je mogoče videti in odboj sončnih žarkov z avtomobila. Prav tako sem dodala še malo učinka megle da poudarimo in simuliramo bolj realistično delovanje sonca. Izgleda tako:





Izrisovanje parkirnih črt:

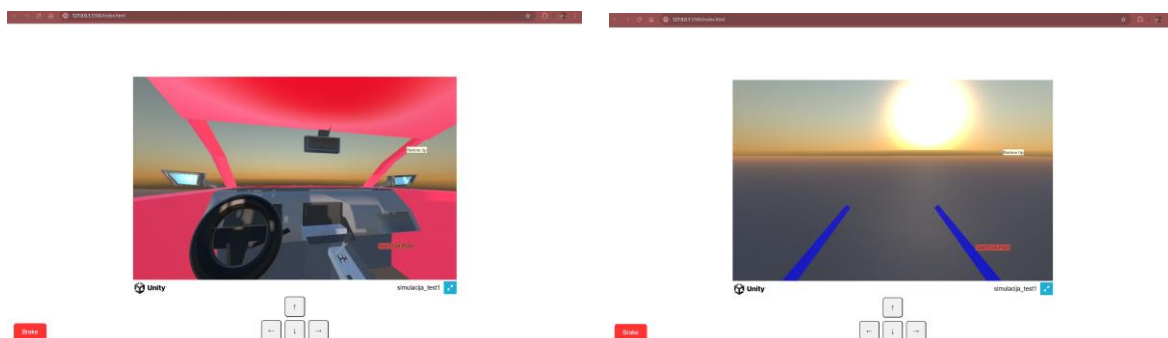
Dodala sem še logiko za izrisovanje parkirnih črt s pomočjo LineRender. Ta bo prejemala informacije o koordinatah označenih oglišč parkirnih črt, ki jih zazna naš model, ter jih nato izrisala. Spodaj je slika ki ponazarja kako model zazna črto in kako jo program nato izriše. Prejete 2D koordinate se normalizirajo iz zaslonskih koordinat in se pretvorijo v 3D world koordinate v območju $[-50, 50]$. Iz vsake četverice koordinat skripta nariše pravokotno parkirno črto s pomočjo LineRenderer komponente.



Eksportiranje drugega dela simulacije iz Unity v WebGL:

Na koncu sem naredila še eksportiranje simulacije iz Unity v WebGL in postavila enostavno komunikacijo med JavaScript in Unity, tako da lahko kontroliramo

spremenljivke. To bo predvsem služilo v nadaljevanju našega projekta, kjer bomo integrirali senzorje za kontrolo obračanja volana in parkirnih črt.



Kjara:

Pridobivanje in posredovanje podatkov

V okviru projekta sem razvila rešitev za pridobivanje, obdelavo in prenos slikovnih podatkov med senzorjem, strežniškim delom sistema in odjemalsko aplikacijo. Glavni cilj je bil zagotoviti zanesljiv prenos podatkov v realnem času ter pripraviti rezultate v obliki, primerni za nadaljnjo uporabo.

Na strani senzorja se zajete slike najprej pretvorijo v sivinski format in nato stisnejo z lastnim kompresijskim algoritmom. S tem se zmanjša količina podatkov, ki se prenašajo po omrežju, kar izboljša odzivnost celotnega sistema. Kompresirani slikovni podatki se nato prek WebSocket povezave pošljejo na strežnik.

Na strežniški strani se prejeta slika dekompresira in pripravi za obdelavo. Nad sliko se izvede model strojnega učenja, ki zaznava parkirne črte. Rezultat zaznavanja so koordinate zaznanih elementov, ki natančno opisujejo njihovo lego v slikovnem prostoru.

Dobljeni rezultati se zapišejo v JSON strukturo, ki omogoča enostavno in standardizirano izmenjavo podatkov. Takšna oblika je posebej primerna za nadaljnjo uporabo v različnih odjemalskih aplikacijah.

Implementirala sem tudi posredovanje JSON rezultatov do Unity WebGL oziroma spletnega odjemalca. Moja naloga je bila zagotoviti pravilno pridobivanje, obdelavo in prenos podatkov, medtem ko vizualni prikaz in izris rezultatov nista bila del moje implementacije.

Za testiranje delovanja sistema sem pripravila poseben testni odjemalec, ki simulira delovanje senzorja. Ta omogoča preverjanje celotnega toka podatkov – od zajema slike, kompresije, obdelave do uspešnega prenosa rezultatov.

MERJENJE FPS IN ČASA UPODABLJANJA

Resolution	FPS	FrameTime(ms)
1920x1080	113,9	8,77
2560x1440	139,96	7,14
3840x2160	174,52	5,73

Po naših meritvah lahko vidimo da povprečni FPS se povečuje z višjo resolucijo, kar je nepričakovano, saj po navadi višja resolucija zmanjšuje FPS. Predvidevamo, da je do tega prišlo ker FPS ni omejen z grafiko, v našem primeru sumimoo, da Unity včasih poveča FPS pri višjih resolucijah, saj lahko te vplivajo na to kako motor upravlja s okvirji in časovnimi intercali --> Frame pacing.

FrameTime se zmanjšuje z višjo resolucijo iz česar lahko sklepamo, da Unity porabi manj časa za en frame pri višjih resolucijah.

Glede na te grafe lahko rečemo, da je simulacija grafično nezahtevna, torej CPU in GPU niso preobremenjeni.

- Simulacija je trenutno zelo lahka za sistem, saj se FPS povečuje z višjo resolucijo, kar pomeni da grafična kompleksnost scene ni previsoka.
- Ozka grla trenutno niso grafična, saj GPU/CPU brez težav obdelata vse okvirje.

